Pipeline para Redes Robustas

https://gemini.google.com/app/78227f19905384f9

User prompt: Dame un pipeline para encontrar redes robustas de alta conectividad:

Response: Para diseñar o identificar **redes robustas con alta conectividad**, es necesario un enfoque integral que combine métricas de teoría de grafos, simulación de ataques y algoritmos de optimización estructural.

A continuación, te presento un pipeline estructurado en 5 etapas principales, acompañado de un bloque de código en Python utilizando **NetworkX** para ponerlo en práctica.

1. Representación y Definición del Espacio de Redes

El primer paso es modelar el sistema como un grafo $G = (V, E)$, donde V son los nodos (regiones, agentes, routers) y E son las aristas (conexiones).

- **Entrada:** Matriz de adyacencia o lista de aristas (pueden ser redes empíricas, como conectomas cerebrales, o modelos sintéticos como *Small-World* o *Scale-Free*).
- **Restricciones:** Definir límites de costo o densidad de conexiones (por ejemplo, mantener el número total de aristas por debajo de un umbral E_{max}).

2. Cuantificación de Conectividad y Robustez

Para evaluar qué tan robusta es una red antes de aplicar modificaciones, se calculan múltiples métricas complementarias:

- **Conectividad algebraica (Valor de Fiedler):** El segundo valor propio menor de la matriz Laplaciana del grafo. Cuanto mayor sea, más difícil será desconectar la red.
- **Conectividad de vértices y aristas:** Número mínimo de nodos o aristas que deben eliminarse para desconectar el grafo.
- **Conectividad natural:** Una medida basada en el espectro de la matriz de adyacencia que cuantifica la cantidad de caminos cerrados alternativos.
- **Coeficiente de clustering y longitud de camino:** Evalúan la eficiencia en la transferencia de información local y global.

3. Simulación de Ataques y Percolación

Una red robusta no solo debe estar bien conectada en condiciones ideales, sino resistir la pérdida de elementos:

- **Ataques aleatorios (Fallos):** Eliminación estocástica de nodos para medir la tolerancia a fallos operativos.
- **Ataques dirigidos (Ataques intencionales):** Eliminación de nodos en orden decreciente de grado, centralidad de intermediación (*betweenness*) o centralidad de autovector.
- **Métrica de salida:** Monitorear el tamaño del componente gigante (*Giant Component*) en función de la fracción de nodos eliminados.

4. Optimización Estructural (Búsqueda de Máxima Robustez)

Si el objetivo es diseñar o modificar una red para maximizar su robustez bajo restricciones:

- **Estrategias de recableado (*Rewiring*):** Intercambio de aristas para incrementar el valor de Fiedler sin aumentar el costo total de la red.
- **Optimización heurística o evolutiva:** Algoritmos genéticos o ascenso de colinas (*Hill Climbing*) donde la función de costo a maximizar es la conectividad algebraica o la resistencia efectiva promedio.

5. Implementación en Python (Ejemplo Base)

El siguiente script en Python utiliza **NetworkX** para calcular el valor de Fiedler y simular la robustez frente a ataques dirigidos:

```
import networkx as nx
import matplotlib.pyplot as plt

def evaluar_robustez_red(G):
    # 1. Calcular conectividad algebraica (Valor de Fiedler)
    fiedler_val = nx.algebraic_connectivity(G, weight=None, method='lanczos')

    # 2. Conectividad de nodos
    node_conn = nx.node_connectivity(G)

    print(f"Conectividad Algebraica (Fiedler): {fiedler_val:.4f}")
    print(f"Conectividad de Nodos: {node_conn}")

    # 3. Simulación de ataque dirigido (removiendo nodos por centralidad de intermediación)
    centrality = nx.betweenness_centrality(G)
    nodes_ordered = sorted(centrality, key=centrality.get, reverse=True)

    sizes_giant = []
    g_copy = G.copy()

    for node in nodes_ordered:
        if g_copy.number_of_nodes() == 0:
            break
        g_copy.remove_node(node)
        if g_copy.number_of_nodes() > 0:
            largest_cc = max(nx.connected_components(g_copy), key=len)
            sizes_giant.append(len(largest_cc) / G.number_of_nodes())
        else:
            sizes_giant.append(0)

    return fiedler_val, sizes_giant

# Ejemplo de uso con una red de ejemplo (Barabási-Albert)
G_synthetics = nx.barabasi_albert_graph(n=100, m=3)
fiedler, resistencia = evaluar_robustez_red(G_synthetics)
```

¿Estás buscando aplicar este pipeline a un dominio específico (como redes de neuroimagen funcional, infraestructura física o grafos abstractos) o tienes alguna restricción particular de tamaño y densidad de nodos?

User prompt: Como redes de neuroimagen funcional

Response: Aplicar este pipeline a **redes de neuroimagen funcional (conectomas funcionales)** requiere tener en cuenta las propiedades biológicas y metodológicas del cerebro: el costo metabólico de las conexiones (*wiring cost*), la presencia de correlaciones negativas, la organización modular y la existencia de *hubs* altamente conectados (como los de la *Default Mode Network* o la red frontoparietal).

A continuación, adaptamos cada etapa del pipeline al análisis de redes cerebrales funcionales (fMRI en estado de reposo, por ejemplo):

1. Representación del Conectoma Funcional

- **Nodos (V):** Regiones de interés (ROIs) definidas mediante un atlas anatómico o funcional (ej. Schaefer, AAL, Power).
- **Aristas (E):** Cuantificadas típicamente mediante correlación de Pearson, correlación parcial, o covarianza regularizada (espacio tangente) entre las series de tiempo BOLD de las regiones.
- **Manejo de pesos y signo:**

- Las redes cerebrales suelen ser **ponderadas** (conservando los pesos de correlación) o **umbralizadas** (manteniendo una densidad de aristas fija, ej. $10\%–20\%$).
- Las correlaciones negativas a menudo se descartan, se convierten en valor absoluto, o se analizan en subgrafos separados, ya que su significado biológico en la señal BOLD sigue siendo debatido.

2. Métricas de Conectividad y Robustez Cerebral

En neurociencia de sistemas, la robustez se evalúa combinando la teoría de grafos clásica con propiedades de organización cerebral:

- **Eficiencia Global y Local:** Miden la capacidad de integración a larga distancia (paralelamiento de la información) y la segregación (procesamiento especializado local).
- **Organización *Rich-Club*:** Evalúa si los nodos con mayor grado (los *hubs* cerebrales) están densamente interconectados entre sí, formando una "columna vertebral" de alta resistencia estructural y funcional.
- **Conectividad Algebraica (Fiedler):** Al igual que en redes generales, cuantifica qué tan difícil es seccionar el cerebro funcionalmente en comunidades desconectadas.
- **Modularidad (Q):** Mide qué tan bien se divide la red en módulos funcionales densos (sistemas sensoriales, motor, cognitivo), lo cual aporta resiliencia frente a fallos locales.

3. Simulación de Lesiones Cerebrales (Ataques Dirigidos a Hubs)

A diferencia de internet o redes de transporte, en el cerebro los ataques dirigidos no son aleatorios; imitan patologías (como neurodegeneración o accidentes cerebrovasculares) que impactan preferentemente a los *hubs* centrales:

- **Estrategia de lesión:** Ordenar los nodos de mayor a menor centralidad (grado, *betweenness* o coeficiente de participación) y eliminarlos secuencialmente.
- **Métrica de degradación:** Medir cómo decae la **eficiencia global** de la red o cómo se fragmenta el componente gigante en función del porcentaje de regiones lesionadas. Las redes con alta robustez muestran una degradación suave y no un colapso catastrófico inmediato.

4. Restricciones Biológicas y Optimización

- **Compromiso Costo-Eficiencia:** El cerebro optimiza la conectividad maximizando la eficiencia de procesamiento mientras minimiza la distancia física y el costo metabólico de las conexiones axonales (estimado frecuentemente mediante la matriz de conectividad estructural DTI/tractografía).
- **Modelado de Perturbaciones:** Simular si la adición de conexiones estratégicas (puentes virtuales entre módulos) incrementa la robustez de la red funcional sin violar restricciones de costo.

5. Implementación en Python (Conectoma Funcional)

El siguiente script simula una matriz de correlación funcional a partir de series de tiempo sintéticas, calcula la conectividad algebraica y evalúa la resiliencia de la red frente a la eliminación progresiva de sus *hubs*:

```
import numpy as np
import networkx as nx

def analizar_robustez_cerebral(time_series, umbral_densidad=0.15):
    # 1. Calcular matriz de correlación funcional (FC)
    corr_matrix = np.corrcoef(time_series)
    np.fill_diagonal(corr_matrix, 0)

    # 2. Construir grafo ponderado umbralizado (manteniendo top_k por ciento de aristas)
    flat_weights = corr_matrix[np.triu_indices_from(corr_matrix, k=1)]
    threshold = np.percentile(flat_weights, 100 * (1 - umbral_densidad))
```

```
    adj_matrix = np.where(corr_matrix >= threshold, corr_matrix, 0)
    G = nx.from_numpy_array(adj_matrix)

    # Eliminar posibles nodos aislados resultantes del umbral
    G.remove_nodes_from(list(nx.isolates(G)))

    if G.number_of_nodes() == 0:
        print("La red está vacía con este umbral.")
        return None

    # 3. Métricas base
    fiedler = nx.algebraic_connectivity(G, weight='weight', method='lanczos')
    global_eff = nx.global_efficiency(G)
    print(f"Conectividad Algebraica (Fiedler): {fiedler:.4f}")
    print(f"Eficiencia Global Inicial: {global_eff:.4f}")

    # 4. Simulación de lesión dirigida a hubs (por Grado Ponderado)
    degrees = dict(G.degree(weight='weight'))
    nodes_ordered = sorted(degrees, key=degrees.get, reverse=True)

    resiliencia_curva = []
    g_copy = G.copy()
    n_inicial = g_copy.number_of_nodes()

    for node in nodes_ordered:
        if g_copy.number_of_nodes() <= 1:
            break
        g_copy.remove_node(node)
        if g_copy.number_of_nodes() > 1:
            eff = nx.global_efficiency(g_copy)
            resiliencia_curva.append(eff / global_eff)
        else:
            resiliencia_curva.append(0.0)

    return fiedler, resiliencia_curva

# Simulación de ejemplo: 50 regiones (ROIs) con 200 puntos temporales de fMRI
np.random.seed(42)
datos_fmri_sinteticos = np.random.randn(50, 200)
fiedler_val, curva_resistencia = analizar_robustez_cerebral(datos_fmri_sinteticos)
```

¿Estás trabajando directamente con datos reales de fMRI (por ejemplo, en formato NIfTI procesados con `fMRIPrep` o `CONN`) para extraer las series de tiempo de un atlas específico?

User prompt: Conn

Response: Integrar este pipeline de robustez con **CONN Toolbox** es directo, ya que CONN cuenta con un módulo robusto de **Teoría de Grafos** y exporta las matrices de conectividad funcional listas para análisis avanzados en MATLAB o Python.

A continuación, se detalla cómo conectar las métricas de robustez y análisis de redes con el flujo de trabajo típico de CONN:

1. Módulo de Teoría de Grafos Nativo en CONN

CONN calcula automáticamente una amplia gama de métricas topológicas a nivel global y nodal (tanto en la GUI como mediante scripts de lotes *batch*).

- **Umbralización por Costo (*Cost Thresholding*):** CONN evalúa las redes en un rango de densidades de conexión (por ejemplo, del 10% al 30% de las aristas más fuertes) para evitar que los resultados dependan de un único umbral arbitrario.
- **Métricas disponibles:**

- *Eficiencia global y local*: Indican la capacidad de integración y segregación.
- *Grado y Centralidad de Intermediación (Betweenness)*: Identifican los *hubs* cerebrales.
- *Modularidad (Q)* y *Coeficiente de Clustering*.

2. Exportación de Datos para Simulación de Ataques (Percolación)

Aunque CONN calcula la centralidad y la eficiencia estándar, **no simula por defecto la degradación de la red ante la eliminación progresiva de nodos** (ataques dirigidos a hubs). Para esto, se pueden extraer las matrices de conectividad directamente desde los archivos de resultados de CONN:

- **Ubicación de las matrices:** En tu directorio de proyecto de CONN, dentro de la carpeta `results/firstlevel/`, encontrarás los archivos MAT correspondientes a las conectividades (por ejemplo, `resultsConnectivity_Condition001.mat`).
- **Variables clave en el archivo MAT:**
  - Z: La matriz de correlación transformada a Fisher-z para cada sujeto.
  - `names`: Las regiones de interés (ROIs) del atlas seleccionado.

3. Script en MATLAB para Calcular la Conectividad Algebraica (Fiedler) desde CONN

Si prefieres procesar las matrices exportadas de CONN dentro de MATLAB para obtener el valor de Fiedler o simular la caída de eficiencia al lesionar los *hubs*, puedes utilizar el siguiente fragmento:

```
% Cargar resultados de primer nivel de CONN para un sujeto específico
load('resultsConnectivity_Condition001.mat', 'Z', 'names');

% Supongamos que tomamos la matriz de conectividad del primer sujeto
sujeto_idx = 1;
fc_matrix = Z(:,:,sujeto_idx);

% Limpiar la diagonal y convertir Fisher-z a correlación (o mantener z)
fc_matrix(isnan(fc_matrix)) = 0;
fc_matrix = tanh(fc_matrix); % Opcional: volver a correlación de Pearson
fc_matrix(1:size(fc_matrix,1)+1:end) = 0;

% Umbralizar por densidad (ej. conservar el top 15% de conexiones positivas)
densidad = 0.15;
vector_valores = fc_matrix(triu(true(size(fc_matrix)), 1));
umbral = prctile(vector_valores, 100 * (1 - densidad));

adj_matrix = fc_matrix .* (fc_matrix >= umbral);

% Construir grafo en MATLAB
G = graph(adj_matrix, 'omittedges');

% 1. Calcular Conectividad Algebraica (Valor de Fiedler)
L = laplacian(G);
autovalores = eig(full(L));
fiedler_val = autovalores(2); % El segundo autovalor menor
fprintf('Conectividad Algebraica (Fiedler): %.4f\n', fiedler_val);

% 2. Simulación de ataque dirigido a hubs (por grado ponderado)
grados = centrality(G, 'degree', 'Importance', G.Edges.Weight);
[~, orden_nodos] = sort(grados, 'descend');

eficiencias = zeros(length(orden_nodos), 1);
G_temp = G;

for i = 1:length(orden_nodos)-1
    % Remover nodo crítico
    G_temp = rmnode(G_temp, orden_nodos(i));
```

```
    if numnodes(G_temp) > 1
        eficiencias(i) = efficiency(G_temp, 'global');
    else
        eficiencias(i) = 0;
        break;
    end
end
```

¿Prefieres realizar este análisis de robustez y simulación directamente mediante scripts en MATLAB aprovechando las matrices de CONN, o te interesa más configurar el módulo de teoría de grafos interactivo dentro de la interfaz gráfica de CONN?
